

Striver SDE Sheet - Recursion Revision Notes

Short DSA revision notes for Recursion problems

Use this as a quick last-minute revision sheet. Focus on the pattern, decision rule, and complexity.

Core Patterns

Pattern	Used In
Pick / Not Pick	Subset Sums, Combination Sum I
Sort + skip duplicates	Subsets II, Combination Sum II
Partition recursion	Palindrome Partitioning
Factorial grouping	K-th Permutation Sequence

Problem-wise Notes

1. Subset Sums

Problem	Generate sums of all possible subsets.
Approach	At every index, make two recursive calls: pick the current element or do not pick it.
Steps	1. Start with index 0 and sum 0. 2. Pick current element and add it to sum. 3. Do not pick current element. 4. When index reaches n, store the sum.
Key Idea	Every element has two choices: include or exclude.
Complexity	Time: $O(2^n)$, Space: $O(n)$ recursion stack

2. Subsets II

Problem	Generate all unique subsets when the array contains duplicates.
Approach	Sort the array and skip duplicate elements at the same recursion level.
Steps	1. Sort the array. 2. Add the current subset to the answer. 3. Loop from current index. 4. Skip $\text{nums}[i]$ if $i > \text{index}$ and $\text{nums}[i] == \text{nums}[i - 1]$. 5. Pick, recurse, then backtrack.
Key Idea	Sorting groups duplicates together, and level-wise skipping avoids duplicate subsets.
Complexity	Time: $O(2^n * n)$, Space: $O(n)$

3. Combination Sum I

Problem	Find combinations whose sum equals target. Same element can be reused.
Approach	Use pick/not-pick recursion. If you pick an element, stay at the same index because reuse is allowed.
Steps	1. If target becomes 0, store current combination. 2. If current element $\leq$ target, pick it and recurse with same index. 3. Do not pick it and move to next index. 4. Backtrack after recursive call.
Key Idea	Reuse allowed means the pick call keeps the same index.
Complexity	Time: Exponential, Space: $O(\text{target})$ approximately

4. Combination Sum II

Problem	Find unique combinations whose sum equals target. Each element can be used once.
Approach	Sort the array, use loop-based recursion, and skip duplicates at the same level.
Steps	1. Sort the array. 2. Loop from current index. 3. Skip duplicates using $i > \text{index}$ condition. 4. If current element $>$ target, break.

	5. Pick element and recurse with $i + 1$.
Key Idea	Use each index once and skip duplicate branches.
Complexity	Time: Exponential, Space: $O(n)$

5. Palindrome Partitioning

Problem	Partition a string so every substring in the partition is a palindrome.
Approach	Try every possible cut. Continue recursion only when the chosen substring is a palindrome.
Steps	1. Start from index 0. 2. Try substrings from index to i. 3. If substring is palindrome, add it. 4. Recurse from $i + 1$. 5. Backtrack after recursion.
Key Idea	Only valid palindrome parts are allowed in the current partition.
Complexity	Time: $O(2^n * n)$, Space: $O(n)$

6. K-th Permutation Sequence

Problem	Find the k th permutation of numbers 1 to n .
Approach	Use factorial blocks instead of generating all permutations.
Steps	1. Store numbers from 1 to n. 2. Convert k to zero-based using $k--$. 3. Use $\text{index} = k / \text{factorial}$ to pick the current number. 4. Remove selected number. 5. Update $k = k \% \text{factorial}$ and continue.
Key Idea	Permutations are grouped in blocks of factorial size.
Complexity	Time: $O(n^2)$, Space: $O(n)$

Final Memory Line

Recursion means: break the problem into choices and solve smaller versions of the same problem.